Final Test Results – Ricart–Agrawala

Distributed Systems · Summary for the README and project report

This document summarizes the final tests performed on the Go implementation of the Ricart–Agrawala distributed mutual exclusion algorithm, using gRPC for communication between three independent nodes.

1. Test setup

- Three independent processes were executed, one for each node: A, B and C.
- Each node acts both as a gRPC server and as a client of the other two nodes.
- Configured local addresses: A (localhost:50051), B (localhost:50052), and C (localhost:50053).
- The Go race detector was enabled using `go run -race`.
- The critical section was simulated by keeping a node inside it for approximately two seconds and logging its entry and exit.

2. Final test: sequential access A → B → C

In the latest shared test, the nodes requested access at different times. The observed entry order was A, followed by B, and finally C. The times below are based on the console logs shared in the conversation.

Node	Observed sequence	Result
A	Requested access at 20:41:17, received the required permissions and entered first. It exited at approximately 20:41:24 and then sent the deferred reply to B.	Access granted; released successfully.
B	Requested access at 20:41:22. After receiving A's deferred permission, B entered and exited at approximately 20:41:26.	Access granted after A.
C	Requested access at 20:41:28, received replies from A and B, and entered last. It exited at approximately 20:41:30.	Access granted after B.

The observed sequence shows no overlap between critical-section executions: A exited before B entered, and B exited before C entered. The logs also show a deferred reply being sent after A released the critical section.

3. Additional test: simultaneous requests and timestamp tie-breaking

In another three-node execution, A, B and C generated requests with the same logical timestamp (1). Since the timestamps were equal, the algorithm used the node identifier as the tie-breaking criterion. The observed order was A → B → C.

- A received replies from B and C and entered the critical section first.
- While A was inside, it deferred replies to requests from B and C.
- B also deferred its reply to C while B's own request had priority.
- A released the critical section at 17:56:50. B then entered; after B released the section, C received the pending permission.
- All three nodes eventually entered and exited. No mutual-exclusion violation was visible in the logs.

4. Race detector results

All three processes were run with `go run -race`. The shared output contained no `WARNING: DATA RACE` messages. Therefore, the race detector did not identify data races during these executions. This is evidence from the tested executions, not a formal proof that the program is free of data races under every possible execution.

5. Evidence related to the assignment requirements

Requirement	Observed evidence	Conclusion for the report
R1 – Specification	Three independent nodes exchange requests and replies through gRPC.	The implementation allows nodes to communicate and request access to the critical section.
R2 – Safety	In the observed executions, nodes entered sequentially; no two nodes were inside the critical section at the same time.	The results are consistent with the mutual-exclusion safety property.
R3 – Liveness	Deferred requests received replies after the higher-priority node left the critical section. A, B and C eventually entered.	In these executions, every node that requested access eventually obtained it.

6. Limitations

The demonstration covers executions in which all three nodes remain operational and local communication is available. This implementation does not provide node-failure tolerance: if a process crashes or stops responding, another node may wait for a reply or its request may fail due to a timeout. Liveness should therefore be understood under the algorithm's usual assumptions, including operational nodes and reliable message delivery.

7. Commands to reproduce the test

Run each command in a separate terminal from the project directory:

```
go run -race . -id=A
go run -race . -id=B
go run -race . -id=C
```

Keep all three processes running so they can receive requests from the other nodes. The demonstration should show the logs from all three terminals.

Note for the submission

This document summarizes the logs and event sequences shared in the conversation; it is not a verbatim console transcript. For the report's log appendix, paste the original complete logs from all three terminals.